Многофайловые программы

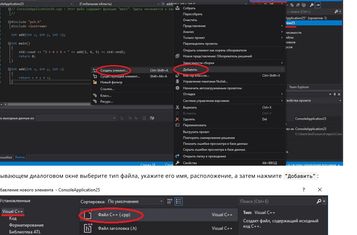
Как только программы становятся больше, их следует разбивать на несколько файлов (в целях удобства и улучшения функциональности). Одним из преимуществ использования IDE является легкость в работе с ними количеством файлов. Мы уже знаем, как создавать и компилировать однофайловые проекты, добавление новых файлов не составит труда.

Оглавление:

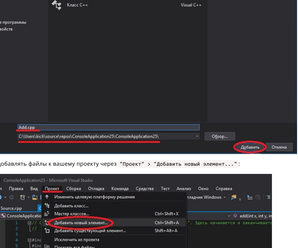
1. Многофайловые проекты в Visual Studio
2. Многофайловые проекты в GCC/G++
3. Пример многофайловой программы
4. Что-то пошло не так!
5. Тест

Многофайловые проекты в Visual Studio

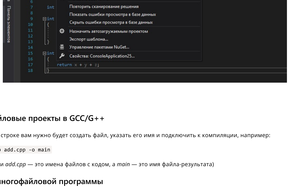
В Visual Studio щелкните правой кнопкой мыши по имени вашего проекта в "Обозревателе решений", затем "Добавить" > "Создать элемент...":



Во всплывающем диалоговом окне выберите тип файла, укажите его имя, расположение, а затем нажмите "Добавить":



Также вы можете добавлять файлы к вашему проекту через "Проект" > "Добавить новый элемент...":



Многофайловые проекты в GCC/G++

В командной строке вам нужно будет создать файл, указать его имя и подключить к компиляции, например:

```
g++ main.cpp add.cpp -o main
```

(*где main.cpp и add.cpp — это имена файлов с кодом, а main — это имя файла-результата*)

Пример многофайловой программы

Рассмотрим следующую программу, которая состоит из двух файлов.

add.cpp:

```
int add(int x, int y)
{
    return x + y;
}
```

main.cpp:

```
#include <iostream>

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << std::endl;
    return 0;
}
```

Попробуйте запустить эту программу. Она не скомпилируется, вы получите следующую ошибку:

```
add: идентификатор не найден
```

При компиляции кода, компилятор не знает о существовании функций, которые находятся в других файлах. Это сделано специально, чтобы функции и переменные с одинаковыми именами, но в разных файлах, не вызывали конфликт имен.

Тем не менее, в данном случае, мы хотим, чтобы main.cpp знал (и использовал) функцию add(), которая находится в add.cpp. Для предоставления доступа main.cpp к функциям add.cpp, нам нужно использовать **предварительное объявление:**

```
#include <iostream>

int add(int x, int y); // мы можем для того, чтобы main.cpp знал, что функция add() определена в другом месте

int main()
{
    std::cout << "The sum of 3 and 4 is: " << add(3, 4) << std::endl;
    return 0;
}
```

Теперь, когда компилятор будет компилировать main.cpp, он будет знать, что такое add(). Попробуйте запустить эту программу еще раз.

Что-то пошло не так!

Есть много вещей, которые могут пойти не так, особенно, если вы это делаете в первый раз. Главное — не паниковать:

Пункт №1: Если вы получили ошибку от компилятора, что функция add() не определена в main(), то, скорее всего, вы забыли записать предварительное объявление функции add() в main.cpp.

Пункт №2: Если вы получили следующую ошибку от линкера:

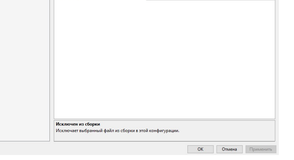
```
unresolved external symbol "int __cdecl add(int,int)" (?add@@YHHH@Z) referenced in function _main
```

то возможных решений есть несколько:

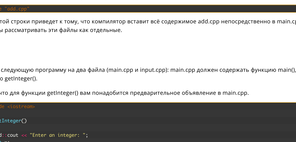
а) Скорее всего, add.cpp некорректно добавлен в ваш проект. Если вы используете Visual Studio или Code::Blocks, то вы должны увидеть add.cpp в "Обозревателе решений" в списке файлов вашего проекта или в панели проекта IDE. Если добавленного файла нет, то щелкните правой кнопкой мыши по вашему проекту и добавьте файл, как это показано выше, а затем повторите попытку компиляции вашего проекта.

б) Вполне возможно, что вы добавили add.cpp к другому проекту.

в) Вполне возможно, что добавленный файл не подключен к компиляции/линкингу. Щелкните правой кнопкой мыши по имени вашего добавленного файла и выберите "Свойства":



Убедитесь, что пункт "Исключен из сборки" оставлен пустым или выбрано значение "Нет":



Пункт №3: Не следует писать следующую строку в main.cpp:

```
#include add.cpp
```

Наличие этой строки приведет к тому, что компилятор вставит всё содержимое add.cpp непосредственно в main.cpp вместо того, чтобы рассматривать эти файлы как отдельные.

Тест

Разделите следующую программу на два файла (main.cpp и input.cpp): main.cpp должен содержать функцию main(), а input.cpp — функцию getInteger().

Помните, что для функции getInteger() вам понадобится предварительное объявление в main.cpp.

```
#include <iostream>

int getInteger()
{
    std::cout << "Enter an integer: ";
    int x;
    std::cin >> x;
    return x;
}

int main()
{
    int x = getInteger();
    int y = getInteger();

    std::cout << x << " + " << y << " is " << x + y << "\n";
    return 0;
}
```

Ответ

input.cpp:

```
#include <iostream>

int getInteger()
{
    std::cout << "Enter an integer: ";
    int x;
    std::cin >> x;
    return x;
}
```

main.cpp:

```
#include <iostream>

int getInteger(); // предварительное объявление функции getInteger()

int main()
{
    int x = getInteger();
    int y = getInteger();

    std::cout << x << " + " << y << " is " << x + y << "\n";
    return 0;
}
```